Reliability Engineering and System Safety 243 (2024) 109821

Contents lists available at ScienceDirect

Reliability Engineering and System Safety

journal homepage: www.elsevier.com/locate/ress

A dual attention LSTM lightweight model based on exponential smoothing for remaining useful life prediction

Jiayu Shi [a], Jingshu Zhong [a], Yuxuan Zhang [a], Bin Xiao [a], Lei Xiao [b], Yu Zheng [a,*]

[a] School Of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai, 200240, China
[b] School Of Mechanical Engineering, Donghua University, Shanghai, 201620, China

ARTICLE INFO

Keywords:
Attention mechanism
Long short-term memory network
Lightweight model
Remaining useful life prediction

ABSTRACT

Accurate remaining useful life (RUL) prediction of degrading systems is crucial to predict failures in advance and develop maintenance plans. As systems degrade gradually over time, sequential degradation feature (SDF) is very important. However, in attention mechanism (AM) based RUL prediction approaches, the sequential operation at each time step is abandoned. Further, these methods are modeled based on numerous parameters, making it difficult to enable timely RUL prediction. Therefore, this paper proposes a dual attention and long short-term memory (LSTM) lightweight model (DA-LSTM). LSTM compensates for the shortcomings of AM in modeling SDF, and exponential smoothing is adopted to train a lightweight model. Specifically, the SDF is divided into aggregated encoding feature (AEF) and aggregated original feature (AOF). AEF is obtained by the encoder which includes a novel soft attention mechanism and an LSTM network. To prevent losing useful information during the encoding process, the second attention layer aggregates the original sensor signal to obtain AOF. Finally, the decoder LSTM network combines AEF with AOF and calculates RUL based on a weighting average method. Extensive experiments are conducted on the C-MAPSS dataset to verify model effectiveness. The results show the superiority of DA-LSTM in prediction accuracy and computational quantity.

1. Introduction

Prognostic and Health Management (PHM) is a critical component of smart manufacturing [1]. It aims to monitor the health condition of critical systems and detect deviations from the normal operating state. An effective PHM method can reduce serious accidents and improve machine safety by providing a real-time diagnosis of the system. One fundamental task in the field of PHM is Remaining Useful Life (RUL) prediction. It aims to estimate the remaining operational time before a system or component fails. The applications of RUL prediction are critical in making proactive maintenance plans and improving operational efficiency. Due to its significance, methods for RUL prediction have been extensively studied.

RUL prediction methods can be divided into three categories, namely knowledge based method, statistical model based method, and data driven method. Knowledge based method is described based on physical equations, materials properties, reliability theories, etc [2–4]. It can accurately model different systems with rich expert knowledge. The statistical model based method concludes patterns in the historical data to infer RUL of the system [5–7]. It is independent of a comprehensive understanding of the system. Nevertheless, the models are constructed based on some strong assumptions and preconditions. By contrast, data driven method does not rely on expert knowledge and strong statistical assumptions, which can be further divided into traditional machine learning method and deep learning method. Traditional machine learning method has been widely adopted for RUL prediction. For instance, Support Vector Machines (SVM) are frequently used in RUL prediction, as it can effectively establish a hyperplane for classification or regression tasks [8]. Another prevalent approach is the Random Forest (RF), which employs an ensemble of decision trees to generate predictions [9]. Traditional machine learning method is computationally efficient. However, it is highly dependent on manual feature construction and is incapable of modeling complex functions.

Compared with traditional machine learning methods, Deep Learning (DL) has shown great potential in modeling complex functions with the increasing amount of available data. DL based method focus on designing an end-to-end approach instead of manual feature construction. It can effectively extract features from the original data with deep neural network structures, such as Fully Connected Neutral Network (FCNN), Recurrent Neutral Network (RNN) [10,11], Convolution Neutral Network (CNN) [12], Graph Convolutional Network (GCN) [13] and Attention Mechanism (AM) [14].

* Corresponding author.
E-mail address: yuzheng@sjtu.edu.cn (Y. Zheng).

https://doi.org/10.1016/j.ress.2023.109821
Received 20 July 2023; Received in revised form 19 October 2023; Accepted 14 November 2023
Available online 18 November 2023

AM based big language models have achieved great success in natural language processing tasks with abundant cloud computational resources and a large number of parameters [15,16]. These models abandon sequential operations like the chain structure of RNN to achieve massive parallel training on multiple cloud Graphical Processing Units (GPU). However, the chain structure is highly effective to extract sequential degradation feature (SDF). In recent years, models which completely rely on AM have been studied to predict RUL [12,17,18]. Due to the absence of chain structure, these models are incapable of capturing SDF in the running process of devices. As devices deteriorate gradually over time, SDF is very crucial for RUL prediction. Therefore, it is important to incorporate sequential information into AM based RUL prediction models.

Further, as AM based models rely on a large number of parameters to model complex functions, existing AM based RUL prediction approaches overlook the study of lightweight models and pay much attention to improving prediction accuracy. However, lightweight models are very useful to reduce response latency in real industrial RUL prediction tasks, which can contribute to detecting failures in time and thus reducing maintenance costs. For example, cloud–edge collaborative RUL prediction is a promising paradigm to make full use of computing resources and reduce response latency [19,20]. However, large-scale models are still difficult to deploy on edge devices due to limited memory capacity and computing resources [21]. This necessitates the development of lightweight models to enable timely response in RUL prediction.

To solve the above two problems, this paper proposes a lightweight model with dual AM and LSTM (DA-LSTM) based on exponential smoothing. Specifically, the encoder LSTM is used to extract the SDF. It can compensate for the shortcomings of AM in modeling sequential information. Inspired by the encoder–decoder structure of transformer, the first LSTM acts as the encoder to extract SDF and another one acts as the decoder to calculate RUL. Moreover, exponential smoothing is used to alleviate ambient noise in the original sensor signal, which can make the model focus on the trend of sensor signals instead of minor fluctuations caused by noise. This method enables the model to achieve higher prediction accuracy with fewer parameters, which can contribute to the RUL deployment on the edge side.

The main contributions of this paper are summarized as follows.

- A novel DA-LSTM lightweight model is proposed based on exponential smoothing. To effectively extract SDF and reduce computation quantity, two different forms of AM are designed to aggregate the features.
- The encoder LSTM is utilized to compensate for the shortcomings of AM in capturing SDF. The decoder LSTM is decomposed into a single time step to further reduce computation quantity and compute RUL.
- Models with different network configurations are widely searched to show the positive effect of exponential smoothing. Based on a suitable smooth rate, lightweight models with higher accuracy and fewer parameters are achieved.
- The proposed RUL prediction approach is verified on the C-MAPSS dataset. Extensive experiments are conducted to prove model effectiveness. The result indicates that the DA-LSTM model can outperform the existing SOTA models in accuracy and the computational quantity is dramatically reduced.

The rest of this paper is organized as follows. Section 3 outlines the proposed RUL prediction framework. Section 4 describes the experimental settings and shows the effectiveness of our method. Section 5 concludes with future work.

2. Related works

2.1. AM based approaches

As for AM based approaches, the most popular model is transformer, which is purely based on the self-dot-product AM. The transformer model consists of the encoder and decoder. Existing works adopted different components of the transformer to predict RUL. Yu et al. [22] took the transformer encoder as the backbone to capture long and short-term dependencies in the time series. To further alleviate noise in the time series, Chen et al. [23] applied a denoising autoencoder to process raw data and the transformer encoder was used to predict RUL of the lithium-ion batteries. A more common practice is to use both the encoder and decoder of transformer. For example, in [24], two encoders were used in parallel to simultaneously extract features of different sensors and time steps. Then the decoder combines two parts of features and outputs RUL. To improve robustness of the model, wavelet threshold denoising [25] and random forest [26] were used for preliminary feature extraction. Based on the preprocessed features, the transformer was adopted to predict RUL. These AM based models can model complex degradation features. However, AM is incapable of capturing SDF. To alleviate this problem, AM-RNN combined models have been studied.

AM can process long sequences while RNN can capture sequential information. Integration of the two methods is a hot issue for current research. One common practice is to stack and take advantage of different deep learning modules sequentially. Liu et al. [27] proposed a feature attention based approach, which gives larger weights to more important features dynamically. Then the weighted features went through the BGRU and CNN sequentially for further feature extraction. One main advantage of AM is that the attention weight matrix can be visualized for better interpretation. In this regard, theoretical foundation and visualization result was given to prove effectiveness of the AM-BGRU combined approach [28]. Other works proposed variants of the traditional RNN architecture. For example, AM is integrated into the LSTM unit to boost the efficiency of long-term dependencies [29,30]. More relevant approaches to our work are the attention based encoder–decoder RNN structure. In [14], BGRU was used based on the encoder–decoder framework. AM was applied in the decoder BGRU to further mine underlying information in the output sequence. Another work [31] integrated AM in the encoder LSTM to better handle long sequences. The aforementioned models have high RUL prediction accuracy. However, these models are often designed to be deeper and wider, resulting in high computation costs. This hinders the timely RUL prediction and poses challenges for deploying them on edge devices.

2.2. Lightweight approaches

Lightweight models can be deployed on resource-constrained devices such as edge computers and Internet of Things (IoT) devices. It can enable timely RUL prediction with low latency and energy consumption. Most of the existing lightweight research has focused on manual model pruning. This technique aims to reduce size of the model by removing redundant parameters and connections while preserving the performance. It has been adopted to AM based models, CNN based models and RNN based models. For AM based models, Zhang et al. [32] proposed a ProbSparse and LogSparse combined self-attention network. By the sparse connection, the network can focus on vital local regions with lower computational complexity. For CNN based models, the Lightweight Temporal Convolution Network (LTCN) was proposed based on the dilated causal convolution layer [19]. To further balance the prediction accuracy and computation quantity, an LTCN enhanced by physical knowledge was explored [33]. For RNN based models, Kayode et al. manually searched the LSTM network structure parameters and proposed a tagging function to reduce model size [34]. Models optimized by manual pruning are highly interpretable. However, manual pruning is laborious and time-consuming.

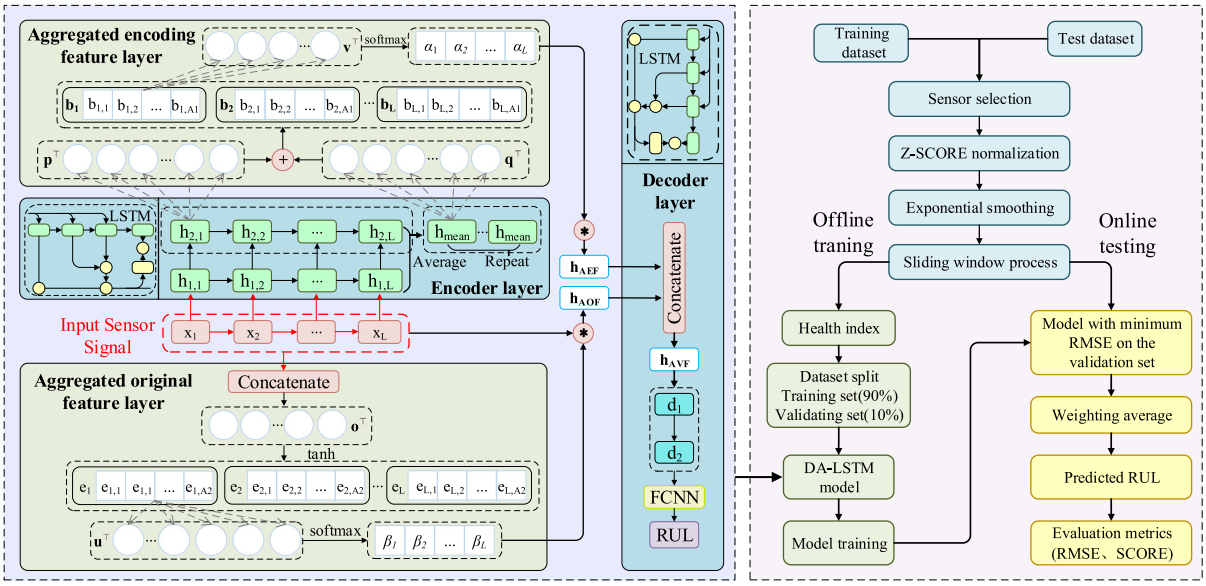


Fig. 1. RUL prediction framework.

To reduce laborious work, there is a current trend toward employing automatic pruning methods, such as Neural Architecture Search (NAS), Reinforcement Learning (RL), and Knowledge Distillation (KD). Researchers have demonstrated their effectiveness in reducing model size and enhancing prediction performance. NAS is the most widely adopted automatic pruning method, which employs techniques like evolutionary algorithms to discover high-performing architectures [35, 36]. For the RL based approaches, it is used to identify redundant elements in the Bi-LSTM model for bearing RUL prediction [37]. For KD based approaches, a multiple-teacher network is compressed into a lightweight student network with high RUL prediction accuracy [21]. To summarize, suitable tailoring strategies are determined without human intervention by continuous iteration. However, these methods require a deep understanding of machine learning algorithms and the searching process is extremely difficult for complex systems.

3. Proposed RUL prediction framework

3.1. Framework overview

The RUL prediction framework is shown in Fig. 1, which is divided into offline training stage and online prediction stage. In the training stage, sensor signals with constant values are eliminated and z-score normalization is adopted to standardize data distribution. Then exponential smoothing is used to remove redundant features and train a lightweight model. On this basis, the sliding window process is adopted to form a consecutive time sequence as the input data, and health index is constructed as the output label. Model with the minimum RMSE on the validation set is applied in the phase of online prediction. The final RUL is obtained with the weighting average method.

The proposed DA-LSTM model mainly contains the following parts, namely the encoder layer, decoder layer, aggregated encoding feature (AEF) layer, and the aggregated original feature (AOF) layer. The overall process is decomposed into the following steps.

- To extract SDF, the original feature is processed by the encoder LSTM. A novel soft AM assigns different weights to the encoded feature and compresses it into a lightweight vector $\mathbf{h}_{\mathbf{AEF}}$.
- To prevent losing useful information in the process of feature encoding, the original feature goes through the aggregated original feature layer. Different weights are assigned to each time step to obtain another lightweight vector $\mathbf{h}_{\mathbf{AOF}}$.

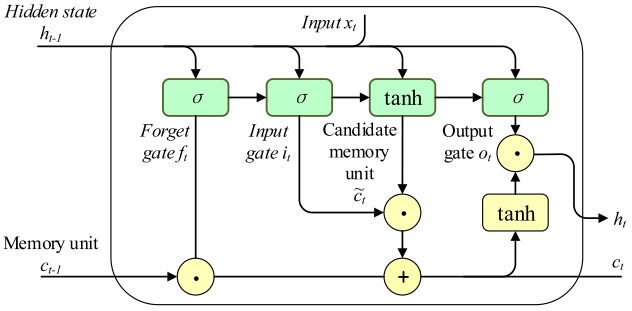


Fig. 2. LSTM unit.

- To further reduce computation quantity, we break up the chain structure of the decoder LSTM and decompose it into a single time step. The decoder LSTM gets the initial hidden state from the encoder.
- $\mathbf{h}_{\mathbf{AEF}}$ and $\mathbf{h}_{\mathbf{AOF}}$ are fused in the feature dimension and then fed into the decoder. Finally, the decoded features are input into a single-layer fully connected neural network. After regression and weighting average, the final RUL is obtained.

3.2. Encoder layer

In the AEF layer, firstly, LSTM is applied to extract SDF. LSTM alleviates the problem of gradient vanishing and gradient explosion by specially designed forget gate, input gate, and output gate. As Eqs. (1)–(6) shows, $\mathbf{f_t}$, $\mathbf{i_t}$, and $\mathbf{o_t}$ are the activated vector of the three gates respectively. σ in Eqs. (1), (2) and (3) is the sigmoid function. *Tanh* in Eq. (4) is the hyperbolic tangent function. σ and *tanh* functions introduce nonlinear characteristics into the LSTM network. $\mathbf{W_f}$, $\mathbf{W_i}$, $\mathbf{W_o}$, $\mathbf{W_a}$, $\mathbf{W_y}$, $\mathbf{V_f}$, $\mathbf{V_i}$, $\mathbf{V_o}$, $\mathbf{V_a}$, $\mathbf{b_f}$, $\mathbf{b_i}$, $\mathbf{b_o}$, $\mathbf{b_a}$, $\mathbf{b_y}$ are parameters that are updated during the training process. $\mathbf{c_t}$ denotes state of the unit at time t, which is used to store memory. Eq. (5) shows that $\mathbf{c_t}$ updates its parameters based on previous unit state $\mathbf{c_{t-1}}$ and candidate memory state $\tilde{\mathbf{c}}_\mathbf{t}$. By contrast, all of the parameters will be updated at each time step in RNN. However, not all $\mathbf{c_t}$ will be updated in LSTM, which is determined by forget gate and input gate. Specifically, as Eq. (5) shows, $\mathbf{f_t}$ determines which part of $\mathbf{c_{t-1}}$ will be forgot, while $\mathbf{i_t}$ determines which part of $\tilde{\mathbf{c}}_\mathbf{t}$ will be added to $\mathbf{c_t}$. Then, $\mathbf{o_t}$ calculates the value of

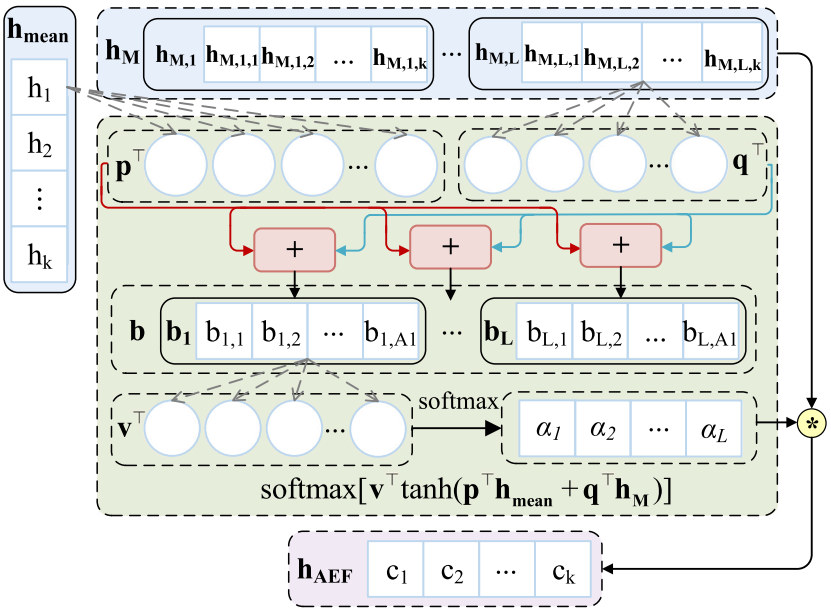


Fig. 3. Aggregated encoding feature layer.

output gate from $\mathbf{h_{t-1}}$ and $\mathbf{x_t}$. Fig. 2 shows the structure of an LSTM unit.

$$\mathbf{f_t} = \sigma(\mathbf{W_f h_{t-1}} + \mathbf{V_f x_t} + \mathbf{b_f}) \tag{1}$$

$$\mathbf{i_t} = \sigma(\mathbf{W_i h_{t-1}} + \mathbf{V_i x_t} + \mathbf{b_i}) \tag{2}$$

$$\mathbf{o_t} = \sigma(\mathbf{W_o h_{t-1}} + \mathbf{V_o x_t} + \mathbf{b_o}) \tag{3}$$

$$\widetilde{\mathbf{c_t}} = tanh(\mathbf{W_a h_{t-1}} + \mathbf{V_a x_t} + \mathbf{b_a}) \tag{4}$$

$$\mathbf{c_t} = \mathbf{f_t} \cdot \mathbf{c_{t-1}} + \mathbf{i_t} \cdot \widetilde{\mathbf{c_t}} \tag{5}$$

$$\mathbf{h_t} = \mathbf{o_t} \cdot tanh(\mathbf{c_t}) \tag{6}$$

The input vector $\mathbf{x} = (\mathbf{x_1}, \ldots, \mathbf{x_t}, \ldots, \mathbf{x_L})$, where $\mathbf{x_t} \in R^n$ denotes an n-dimension vector, $t \in [1, L]$ denotes the time step, n denotes the number of sensors, L denotes the sliding window length. Eq. (7) shows that the encoder LSTM f_{enc} takes the vector $\mathbf{x}$ as input and outputs hidden state $\mathbf{h_m} = (\mathbf{h_{m,1}}, \mathbf{h_{m,2}}, \ldots \mathbf{h_{m,L}})$:

$$\left(\mathbf{h_{m,1}}, \ldots, \mathbf{h_{m,L}}\right) = f_{\text{enc}}\ \left(\mathbf{x_1}, \mathbf{x_2}, \ldots, \mathbf{x_L}; \theta_{enc}\right) \tag{7}$$

where $m \in [1, M]$, M denotes the number of layers of the encoder LSTM, k denotes the number of hidden units in the encoder LSTM, $\theta_{enc} = \left[\mathbf{W_{enc}}, \mathbf{U_{enc}}, \mathbf{b_{enc}}\right]$ denotes learnable parameters of the encoder LSTM.

3.3. Aggregated encoding feature layer

After the SDF is obtained, a novel form of AM is designed. It can further mine useful information in the deep LSTM network and compress it into a lightweight vector. Firstly, AM is composed of three elements, i.e., query, key, and value. In previous AM based research [12,18,38], they are often the same vector, namely self-attention mechanism. But in this paper, query is different from the key–value pair. Specifically, the hidden state $\mathbf{h_M} = (\mathbf{h_{M,1}}, \mathbf{h_{M,2}}, \ldots \mathbf{h_{M,L}})$ at all time steps in the last layer is applied as the key–value pair. $\mathbf{h_M}$ is a two-dimensional vector, of which the two dimensions are time steps and number of hidden units. And then the output vector $\mathbf{h_L} = (\mathbf{h_{1,L}}, \mathbf{h_{2,L}}, \ldots \mathbf{h_{M,L}})$ at the last time step in all the layers is averaged. $\mathbf{h_L}$ is also a two-dimensional vector, of which the two dimensions are number of LSTM layers, number of hidden units. Eq. (8) shows that $\mathbf{h_L}$ is averaged on the first dimension to calculate $\mathbf{h_{mean}}$, which is applied as query in the soft attention mechanism. Due to the average operation, the layer dimension is removed. Consequently, $\mathbf{h_{mean}}$ is a one-dimensional vector, where $\mathbf{h_{mean}} \in R^k$.

$$\mathbf{h_{mean}} = 1/M \cdot \sum_{m=1}^{M} \mathbf{h_{m,L}} \tag{8}$$

$$\mathbf{b} = \mathbf{p}^\top \cdot \mathbf{h_{mean}} + \mathbf{q}^\top \cdot \mathbf{h_M} \tag{9}$$

$$\mathbf{s_{enc}} = \mathbf{v}^\top \tanh(\mathbf{b}) \tag{10}$$

$$\boldsymbol{\alpha} = softmax(\mathbf{s_{enc}}) \tag{11}$$

$$\mathbf{h_{AEF}} = \sum_{t=1}^{L} \alpha_t \mathbf{h_{M,t}} \tag{12}$$

$\mathbf{h_{mean}}$ is applied as query because the hidden state at the last time step contains the most valuable information based on the chain structure of LSTM. Then, the similarity between $\mathbf{h_M}$ and $\mathbf{h_{mean}}$ is measured through the ADDITIVE attention scoring function [39]. As Eq. (9) shows, $\mathbf{b}$ is the element-wise addition sum of $\mathbf{h_M}$ and $\mathbf{h_{mean}}$, A_1 denotes the hidden units in the fully connected layer $\mathbf{p}^\top, \mathbf{q}^\top$. After the attention score s_{enc} is obtained, the attention weights $\boldsymbol{\alpha} = (\alpha_1, \alpha_2, \ldots, \alpha_L)$ is calculated by the softmax function, as Eq. (11) shows. Encoded feature which is more similar to $\mathbf{h_{mean}}$ are given larger weights, which means they are more important. Finally, as Eq. (12) shows, a lightweight vector $\mathbf{h_{AEF}}$ is obtained through the weighted sum of $\mathbf{h_{M,t}}$, where $\mathbf{h_{AEF}} \in R^k$.

Fig. 3 shows the overall calculation process of AEF. LSTM can model the SDF while AM can take advantage of the hidden state in the deep LSTM network. By contrast, traditional methods only pay attention to the hidden state in the last layer, thus losing useful information.

3.4. Aggregated original feature layer

To further extract SDF, the second attention layer is designed to aggregate the original input sequence. Then a new lightweight degradation vector $\mathbf{h_{AOF}}$ is obtained. As Fig. 4 shows, the input vector is $\mathbf{x} =$

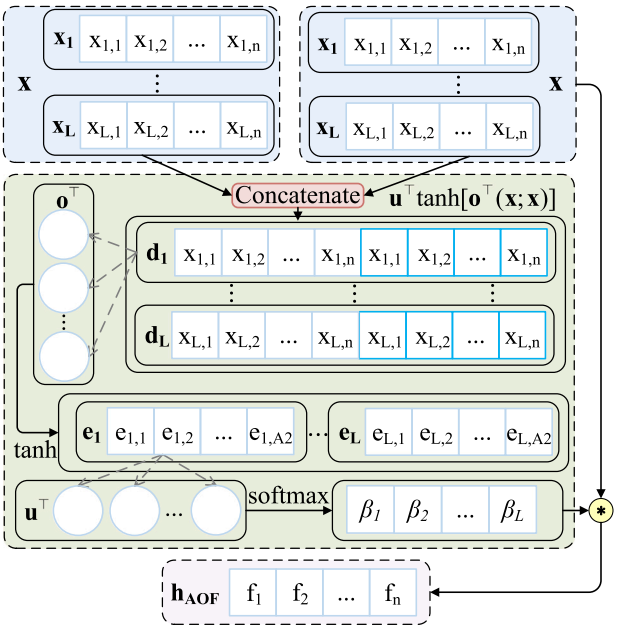


Fig. 4. Aggregated original feature layer.

$(\mathbf{x_1}, \ldots, \mathbf{x_t}, \ldots, \mathbf{x_L})$, where $\mathbf{x_t} \in R^n$, n denotes the number of sensors, t denotes the time steps. Differing from the common *dot* product attention-scoring function, the *concat* scoring function is adopted.Experiments that compare different attention-scoring functions are illustrated in Section 4. Specifically, as Eqs. (13)–(14) shows, two identical vectors $\mathbf{x}$ are concatenated on the sensor dimension to calculate the attention score $\mathbf{s_{ori}}$ of different time steps. Then the two vectors go through two fully connected layers sequentially, i.e., $\mathbf{o}^\top, \mathbf{u}^\top$, where A_2 represents the number of hidden units in $\mathbf{o}^\top$. The attention score $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_t, \ldots, \beta_T)$ is converted into attention weights by the $softmax$ function. Finally, the input information is weighted to get $\mathbf{h_{AOF}}$, where $\mathbf{h_{AOF}} \in R^n$, as Eq. (16) shows.

$$\mathbf{d} = [\mathbf{x}; \mathbf{x}] \tag{13}$$

$$\mathbf{s_{ori}} = \mathbf{u}^\top \tanh(\mathbf{o}^\top \cdot \mathbf{d}) \tag{14}$$

$$\boldsymbol{\beta} = softmax(\mathbf{s_{ori}}) \tag{15}$$

$$\mathbf{h_{AOF}} = \sum_{t=1}^{L} \beta_t \mathbf{x_t} \tag{16}$$

3.5. Decoder layer

Inspired by the encoder–decoder structure of transformer, the second LSTM network is utilized as the decoder to calculate RUL. We break up chain structure of the decoder LSTM and decompose it into a single time step. It is a brand-new attempt and can reduce computation quantity. As Eq. (17) shows, the two lightweight vectors $\mathbf{h_{AEF}}$ and $\mathbf{h_{AOF}}$ are concatenated to calculate the aggregated overall feature $\mathbf{h_{AVF}}$:

$$\mathbf{h_{AVF}} = [\mathbf{h_{AOF}}; \mathbf{h_{AEF}}] \tag{17}$$

where $\mathbf{h_{AEF}} \in R^k$, $\mathbf{h_{AOF}} \in R^n$, $\mathbf{h_{AVF}} \in R^{n+k}$.

Then, $\mathbf{h_{AVF}}$ is input into the decoder LSTM, and the decoded feature $\mathbf{h_D}$ is obtained. As RUL prediction is a regression problem, a single-layer fully connected neural network is utilized to calculate RUL:

$$\mathbf{h_D} = f_{dec}(\mathbf{h_{AVF}}; \boldsymbol{\theta}_{dec}) \tag{18}$$

Table 1
Description of the C-MAPSS dataset.

Dataset description	FD001	FD002	FD003	FD004
Number of engines in the training set	100	260	100	249
Number of engines in the test set	100	259	100	248
Operating conditions	1	6	1	6
Fault modes	1	1	2	2

Table 2
Trend of sensor signals in FD001 and FD003.

Trend type	Sensor number
Upward or downward trend	2,3,4,7,8,9,11,12,13,14,15,17,20,21
Consistent trend	1,5,6,10,16,18,19

where $\mathbf{h_D} \in R^e$, e represents the number of hidden units in the encoder LSTM, $\boldsymbol{\theta}_{\mathbf{dec}} = [\mathbf{W_{dec}}, \mathbf{U_{dec}}, \mathbf{b_{dec}}]$ denotes the parameters of the decoder LSTM.

4. Experiments and results

4.1. Dataset description and data preprocessing

4.1.1. Dataset description

C-MAPSS is a software tool developed by NASA to simulate a real turbofan engine. It is written in the MATLAB Simulink environment to simulate a 90,000-lb thrust engine model. Based on this software, NASA released the C-MAPSS dataset to facilitate the development of more accurate RUL prediction models [40].

As shown in Table 1, the C-MAPSS dataset consists of four sub-datasets. Each contains 21 sensor signals and is further divided into the training set and test set. In the training set, each engine operates normally in the beginning but starts to degrade at a certain time. True RUL labels of the engines are provided. In the test set, the data is incomplete, and the time series terminate at a certain time before the engine degrades. The goal is to predict how many cycles the engine can still run.

The sensor signals exhibit three trends: upward, downward, and consistent, as shown in Table 2. Only 14 sensor signals which always show an upward trend and downward trend on the four sub-datasets are retained, because they can provide more information than the signals with a consistent trend [12,41,42].

4.1.2. Data normalization

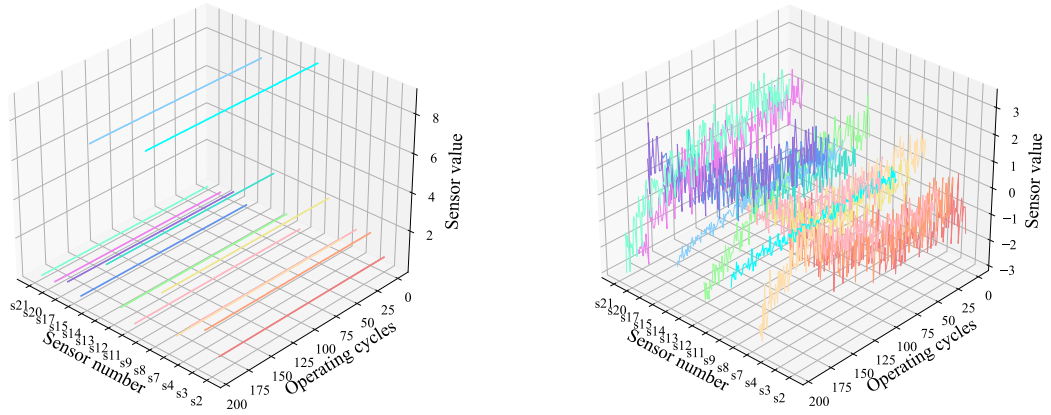
The sensor signals in the C-MAPSS dataset have different units, and the range of sensor values changes greatly. If the original data is directly used for analysis, features with small values will be ignored and it is difficult for the neural network to converge. To scale the data distribution and unify dimensions, Z-SCORE normalization is adopted [10,41]:

$$x'_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j} \tag{19}$$

where $x'_{i,j}$ is the normalized value of the jth feature of the ith sample, $x_{i,j}$ is the original value of the jth feature of the ith sample, μ_j is the mean of the jth feature and σ_j is the standard deviation of the jth feature. The sensor signals are distributed in different intervals before normalization, which hinders subsequent feature learning. By contrast, the data shows a clear changing trend after normalization, as Fig. 5 shows.

4.1.3. Data smoothing

The original sensor signal contains a lot of noise, which may cause the model to learn irrelevant features. The influence of historical sensor values on future values decreases with time intervals. Therefore, a more practical method is to use the weighted average of the observed values



(a) Before normalization (b) After normalization

Fig. 5. Comparison for data normalization.

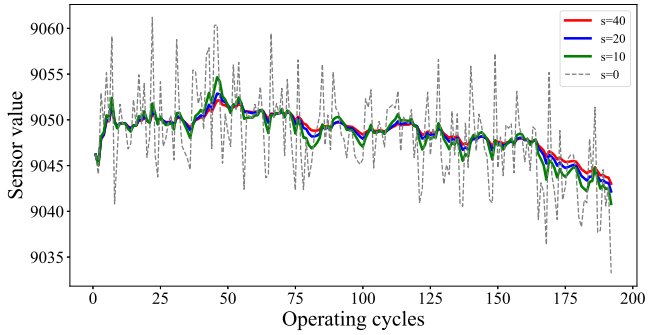


Fig. 6. Comparison between different s.

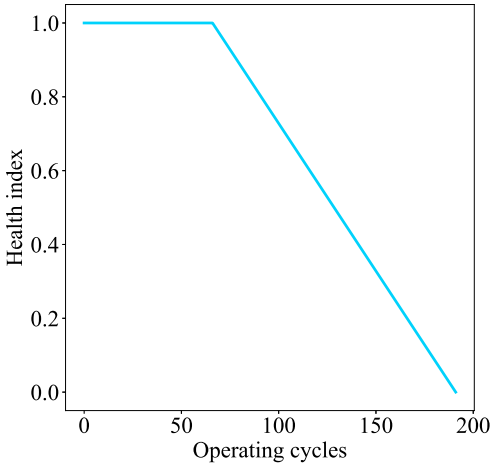


Fig. 7. Health index degradation model.

according to the time order as the predicted value. The exponential smoothing method can satisfy this requirement and has a simple recursive form. It can reduce oscillation and filter out redundant features while retaining the trend of sensor signals, as Eqs. (20)–(22) shows. Points that are closer to the current time are given larger weights. The attenuation factor $\gamma \in [0, 1]$ is used to adjust smoothness. s is used as the smoothing coefficient ($s > 1$). Eq. (22) shows the relationship between γ and s.

$$y_t = \frac{\sum_{i=0}^{t} w_i x_{t-i}}{\sum_{i=0}^{t} w_i} \tag{20}$$

$$w_i = (1-\gamma)^i \tag{21}$$

$$\gamma = \frac{2}{1+s} \tag{22}$$

4.1.4. Health index

The backpropagation algorithm is adopted to update the parameters. This is a supervised learning algorithm, so true RUL needs to be provided in the training stage. The most common models to describe RUL are the linear degradation model and the piecewise linear degradation model. Health Index (HI) is constructed based on the piecewise linear degradation model [43,44], which is a metric that helps researchers estimate the health status of machines [45]. Previous research has shown that constructing an appropriate HI requires rich expertise, which contradicts the original intention of using data-driven methods. Consequently, a simple form of HI is adopted. HI at time t is given as:

$$HI_i = 1 - max(\frac{T_{cp} - EOL_i + t_i}{T_{cp}}, 0) \tag{23}$$

where HI_i denotes HI for the ith engine, T_{cp} denotes time for the change point, t_i denotes the operating cycles for the ith engine, EOL_i (End of Life) denotes the total operating cycles for the ith engine. T_{cp} is set to 125 empirically [12,36]. As Fig. 7 shows, HI is a simple piecewise linear function, whose value goes from 1 to 0 over time. When HI is equal to 1, it indicates the engine is in healthy state. When HI is equal to 0, it indicates the engine is in a degraded state (see Fig. 6).

4.1.5. Sliding window

As Fig. 8 shows, the horizontal axis denotes the sensor dimension while the vertical axis denotes the time dimension. Different features are highlighted in different colors. A sliding window with length L is used to encapsulate the sensor signal sequence. A new sliding window is generated for each time step until the degradation point. $X_i = (x_1, x_2, \ldots, x_l, \ldots, x_{N-L+1})$ represents the sliding window sequence for the ith engine, where $x_l \in R^{L \times n}$, n denotes the number of sensors, L denotes length of the sliding window, N denotes the total time length.

Larger sliding windows can contain more information, which is conducive to extracting features hidden in sensor signal sequences. However, this will reduce the training speed of the model and increase computation quantity. Moreover, the accuracy of the model will decrease when the length of the window increases to a certain threshold [12,41]. The length of the sliding window is set to 30 based on previous research [46], which is also an AM and LSTM based approach. In the test set, a few windows less than 30 in length are filled by cubic spline interpolation.

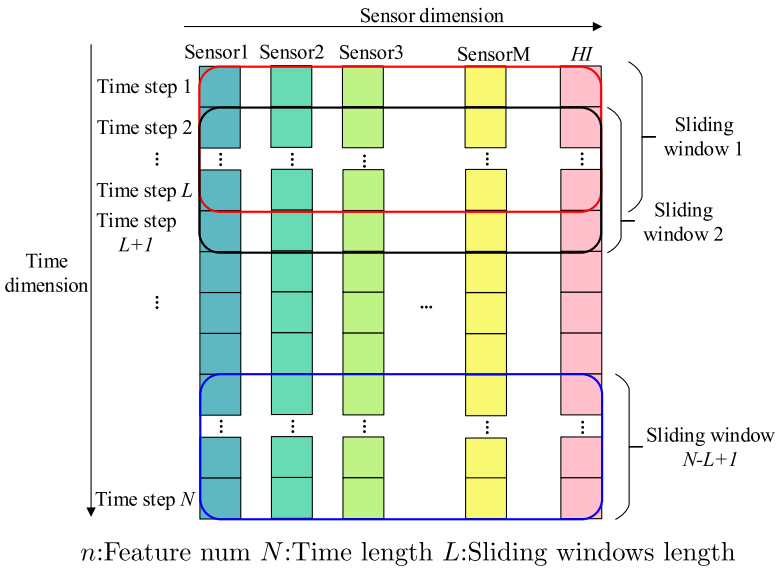


Fig. 8. Sliding window feature extraction.

4.2. Experimental settings and evaluation metrics

4.2.1. Experimental settings

In the process of training, the back-propagation algorithm [47] and the mini-batch gradient decent method is utilized to optimize the parameters. The loss function can be formulized as:

$$Loss = \frac{\sum_{i=1}^{M}(\widetilde{HI}(i) - HI(i))^2}{M} \tag{24}$$

where M denotes the batch size, $\widetilde{HI}(i)$ denotes predicted HI and $HI(i)$ denotes true HI of the ith sample. Our code implementation is available at https://github.com/tortorish/CMAPSS-release.

4.2.2. Weighting average method

The directly estimated RUL contains some noise and oscillation, which will increase the prediction error between the actual RUL and the estimated RUL. To overcome this problem, this paper adopts the weighted average method to denoise the prediction results by taking an average of the first few consecutive data points. The final RUL is calculated by:

$$RUL_m = \left(\sum_{i=m-4}^{m} \delta_i \times RUL_i\right) / \sum_{i=m-4}^{m} \delta_i \tag{25}$$

where RUL_m denotes predicted RUL of the mth engine. δ_i denotes weight of the ith point. In this paper, $\delta_{m-4} = \delta_{m-3} = \cdots = \delta_m = 0.2$.

4.2.3. Evaluation metrics

To evaluate the performance of the model, two functions are adopted. One is the root mean square error (RMSE), and another is the SCORE function [40]. Define $d = RUL_{predict} - RUL_{true}$, namely RUL prediction value minus RUL true value. RMSE and SCORE is calculated through:

$$RMSE = \sqrt{\frac{1}{M}\sum_{i=1}^{M} d_i^2} \tag{26}$$

$$SCORE = \begin{cases} \sum_{i=1}^{M} e^{-(d/a_1)} - 1 (d < 0) \\ \sum_{i=1}^{M} e^{(d/a_2)} - 1 (d \geq 0) \end{cases} \tag{27}$$

where M denotes number of engines, $a_1 = 10$, $a_2 = 13$.

For engine degradation scenarios, it is desirable to detect faults in advance, which means early prediction is preferable to late prediction. If $d < 0$, the predicted value of RUL is less than the true value. A small penalty is given because there is still time for maintenance. By contrast, if $d \geq 0$, the penalty becomes larger because the fault cannot be predicted in advance.

4.3. Analysis of the proposed network structure

In this section, extensive experiments are carried out to configure the attention-scoring functions and structural parameters. The ten cross-validation method is conducted ten times to get the proper network structure. In addition, models can be easily generalized from one sub-dataset to the others [36]. For example, the best architecture for FD004 also shows a good performance on the other sub-datasets. Consequently, the model is trained on FD004 to optimize the structural parameters, which saves us a lot of time.

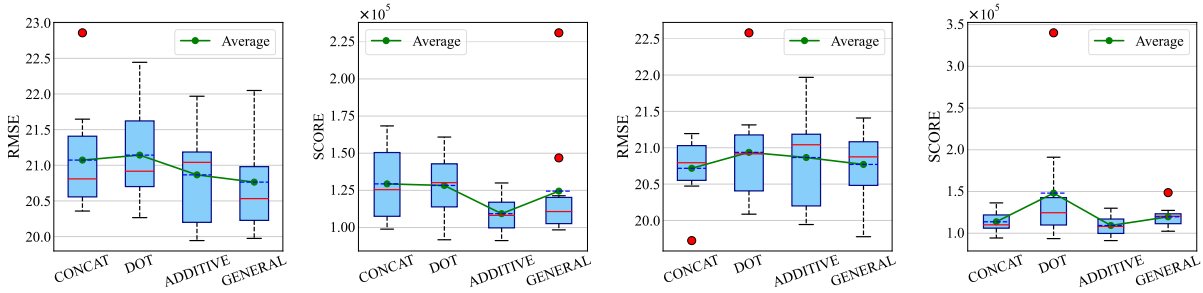
4.3.1. Analysis of the scoring function

Attention-scoring function is a crucial part of AM, which measures the similarity between the key and query. Different scoring functions will make the model show different performances. To better aggregate the SDF obtained by the encoder LSTM, the effect of different scoring functions is analyzed. They were previously adopted in the domain of machine translation [39]. Four forms of attention-scoring functions are written as:

$$\text{score}(\mathbf{p}, \mathbf{q}) = \begin{cases} \mathbf{p}^\top \cdot \mathbf{q} & dot \\ \mathbf{p}^\top \cdot \mathbf{W_v} \cdot \mathbf{q} & general \\ \mathbf{W_v}^\top \tanh(\mathbf{W_p}^\top [\mathbf{p}; \mathbf{q}]) & concat \\ \mathbf{W_v}^\top \tanh(\mathbf{W_p} \cdot \mathbf{p} + \mathbf{W_q} \cdot \mathbf{q}) & additive \end{cases} \tag{28}$$

where $\mathbf{W_v}, \mathbf{W_p}, \mathbf{W_q}$ are renewable parameters.

AM is applied in both the AEF layer and AOF layer. Thus the scoring functions are analyzed respectively. Firstly, the scoring function in the AEF layer is studied. As Fig. 9(a)(b) shows, the ten-cross validation is conducted ten times. The results indicate that the RMSE average of *additive* and *general* is the lowest. However, the SCORE of *general* fluctuates greatly and two outliers appeared in ten times of training, which leads to a bigger average of SCORE. By contrast, the SCORE distribution of *additive* is the most stable and the average of RMSE is also lower. Therefore, the *additive* scoring function is adopted in the AEF layer. Then the scoring function in the AOF layer is studied. As Fig. 9(c)(d) shows, SCORE of *dot* and *general* is larger and there appear two outliers. By contrast, the SCORE of *concat* and *additive* is relatively lower and has little oscillation. In addition, the *concat* scoring function



(a) AEF layer (RMSE) (b) AEF layer (SCORE) (c) AOF layer (RMSE) (d) AOF layer (SCORE)

Fig. 9. Comparison for different scoring functions.

Table 3
Study on structural parameters.

Network structure	RMSE	Layer num	Hidden units	Parameter num
I: E(8)D(8)	21.06	1	8	2741
II: E(16)D(16)	21.29	1	16	6397
III: E(8,8)D(8,8)	20.48	2	8	3893
IV: E(16,16)D(16,16)	20.88	2	16	10749
V: E(8,8,8)D(8,8,8)	21.02	3	8	5045
VI: E(16,16,16)D(16,16,16)	20.86	3	16	15101
VII: E(12,12)D(12,12)	20.74	2	12	6873
VIII: E(10,10)D(10,10)	20.59	2	10	5271
IX: E(8,8)D(8,8)	20.48	2	8	3893
X: E(7,7)D(7,7)	20.93	2	7	3288
XI: E(6,6)D(6,6)	21.48	2	6	2739
XII: E(4,4)D(4,4)	21.36	2	4	1809

also performs better in terms of RMSE. Therefore, the *concat* scoring function is adopted in the AOF.

To summarize, the scoring function of *concat* and *additive* performs better. We infer that this is because these two forms of scoring functions have more learnable parameters ($\mathbf{W_v}, \mathbf{W_p}, \mathbf{W_q}$), which can be updated through the backpropagation process.

4.3.2. Analysis of the structural parameters

To balance model size and prediction accuracy, four network structural parameters need to be determined, including number of LSTM layers of the encoder and decoder (E, D), number of hidden units of (E, D), number of neurons (A_1, A_2) in the AEF layer and the AOF layer respectively. They all have an impact on performance of the model. As the hidden state of the decoder is initialized by the encoder, the LSTM networks of (E, D) have the same number of layers and hidden units. Moreover, the hyperparameters of the LSTM network have few relationships [48], so the best value of each hyperparameter is determined separately.

Firstly, the number of LSTM layers is configured. The notation E(8,8)D(8,8) denotes that both the (E, D) LSTM have two layers and 8 hidden units. As Table 3 shows, models I,II, III, IV are compared firstly. The RMSE decreases significantly as the number of layers is increased from 1 to 2. This indicates that a deeper network can capture more useful information compared with a shallow one. Then, when we compare models III, IV, V, VI. If the layer number is increased continuously, this will not decrease RMSE accordingly while the number of parameters increases evidently. Thus the number of LSTM layers is set to 2.

Then the number of LSTM hidden units needs to be determined. As Table 3 shows, RMSE of model IX is the lowest. However, when the number of hidden units is decreased continuously, RMSE will increase significantly. This indicates that the ability of the model to capture effective features in multi-sensor sequences becomes weaker. Similarly, models VII, VIII are compared. As the number of hidden units is increased continuously, RMSE will not decrease but the model size will increase evidently. Therefore, the number of LSTM hidden units is set to 8.

Finally, the number of units A_1, A_2 in the two attention layers needs to be configured. As the search scope is small, a simple hyperparameter grid search is adopted. The result shows that $A_1 = 28, A_2 = 4$ is the best combination. Then two indicators are applied to evaluate the overall computation quantity: number of parameters and number of floating point operations (FLOPs). Each addition, subtraction, multiplication, and division operation is counted as 1 FLOP, which is a commonly used metric to evaluate the computational quantity of the model. By attention aggregation, the original long sequence is transferred into a short lightweight vector $\mathbf{h_{AOF}}$. Then $\mathbf{h_{AOF}}$ is fed into the decoder LSTM that contains only one time step. Based on this novel design, as Table 4 shows, although there are more parameters in the decoder compared with the encoder, the floating point operations are significantly reduced by breaking up the chain structure of LSTM.

To summarize, Table 5 lists configurations of the proposed method.

4.4. Comparison against state-of-the-art

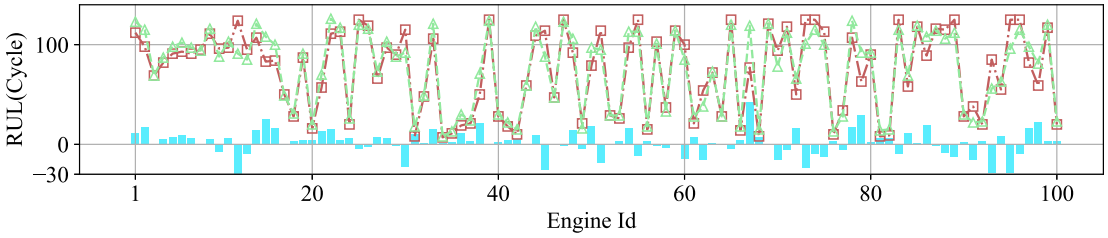
In the training process, the learning rate decay method is adopted. The initial learning rate is set to 0.002 and it decays to 10% of the original every 10 epochs. Additionally, the RMSPOP algorithm is adopted to optimize the parameters and the batch size is 128. Moreover, 90% of engines are set as the training set and 10% of engines are set as the validation set. To prevent the model from falling into a local minimum, different random seeds are utilized to train the model on the four sub-datasets for 50 times respectively. Finally, the model with the least RMSE on the validation set is saved and then tested on the test set.

4.4.1. Comparison for RMSE and SCORE

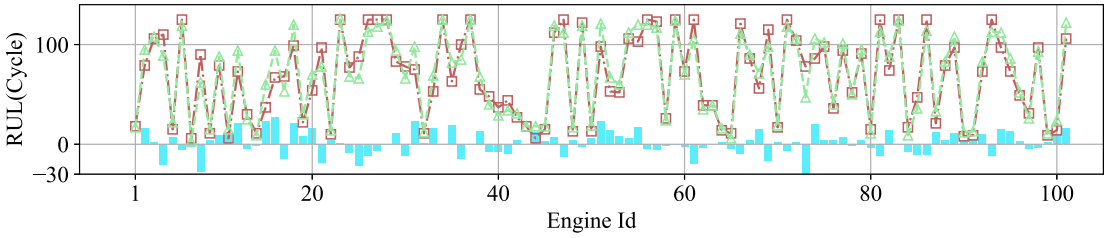
As Table 6 shows, state-of-the-art (SOTA) models are chosen as references to better illustrate the overall performance of the proposed DA-LSTM model. N/A means the information is unavailable. Including four AM-based models [12,18,32,54], four AM-RNN based models [27,28,46,52], three RNN based models [10,49,51] and three RNN-CNN based models [36,50,53]. RMSE and SCORE of the proposed DA-LSTM model on the FD002 and FD004 datasets are significantly better than

Table 4
Parameter statistics.

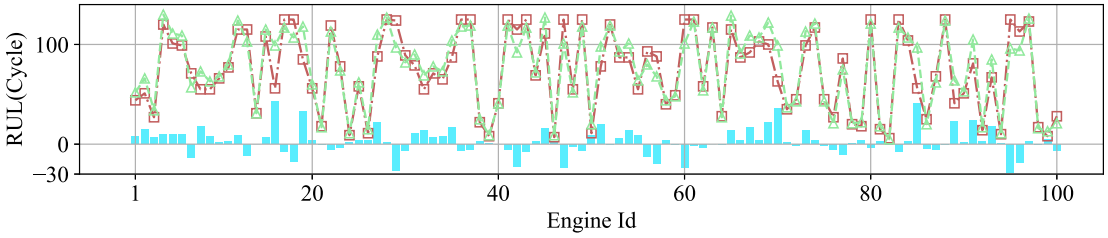
Component	Encoder layer	AEF layer	AOF layer	Decoder layer	Total
Parameter num	1344	477	120	1609	3550
FLOPs	8.83×10^4	2.86×10^4	6.96×10^3	3.47×10^3	1.27×10^5



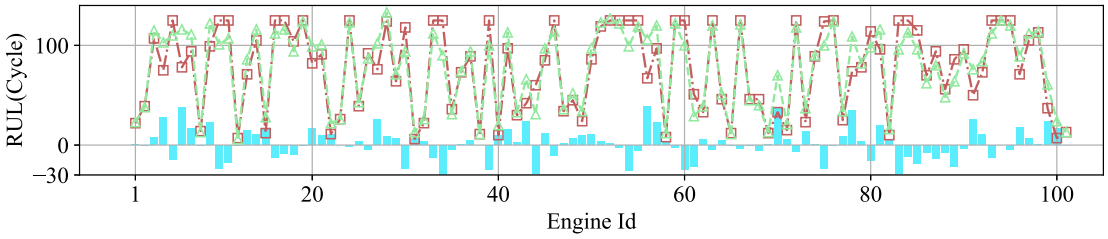
(a) Result on FD001



(b) Result on FD002 (Engine 1-Engine 100)



(c) Result on FD003



(d) Result on FD004 (Engine 1-Engine 100)

Fig. 10. Prediction results on the four sub-datasets.

Table 5
Configurations of the proposed method.

Parameter	Value
Number of LSTM hidden units	8
Number of LSTM layers	2
Number of hidden units in the AEF layer	28
Number of hidden units in the AOF layer	4
Number of hidden units in the fully connected layer	8
Sliding window length	30
RUL change point T_{cp}	125
Smooth rate for FD001/FD002/FD003/FD004	30/40/30/40
Attention scoring function of the AEF layer	*additive*
Attention scoring function of the AOF layer	*concat*

the SOTA. While on the FD001 and FD003 datasets, our model is quite close to the SOTA.

To clearly demonstrate the comprehensive performance of each model under different operating conditions, the average and variance of each algorithm are calculated. As presented in Table 7, the DA-LSTM model exhibits the lowest average and variance of both RMSE and SCORE, thus suggesting that our proposed method provides the best overall performance across the four sub-datasets. Moreover, low variance indicates that the model based on exponential smoothing possesses good generalization ability and robustness. In other words, the accuracy can still be maintained at a high level when the dataset becomes more complex. However, as FD001 and FD003 are relatively simple, the hyperparameters of some models [28,32,52] are optimized on these subdatasets. This results in a high prediction accuracy on FD001 and FD003, but relatively poor performance on FD002 and FD004. Other models [36,54] are optimized on all of the four sub-datasets, which cost a lot of computing resources. By contrast, network

Table 6
Comparison of different models.

Methods	RMSE				SCORE			
	FD001	FD002	FD003	FD004	FD001	FD002	FD003	FD004
D-LSTM [10], 2017	16.14	24.49	16.18	28.17	338	4450	852	5550
BS-LSTM [49], 2018	14.89	26.86	15.11	27.11	481	7982	493	5200
BL-CNN [50], 2019	13.18	19.09	13.75	20.97	302	1558	381	3859
RF-LSTM [51], 2020	16.87	23.91	17.89	25.49	N/A	N/A	N/A	N/A
AM-LSTM [46], 2021	14.53	N/A	N/A	27.08	322.44	N/A	N/A	5649.14
CNN-LSTM [36], 2021	11.56	17.67	12.98	20.19	247	1743	808	3051
AM-BGRU [27], 2021	12.42	19.43	13.39	21.50	226	1492	227	3392
GCU-Transformer [18], 2021	**11.27**	22.81	11.42	24.86	N/A	N/A	N/A	N/A
NTM [52], 2022	12.53	27.04	13.73	28.11	210	6000	270	4800
BIGRU-TSAM [28], 2022	12.56	18.94	12.45	20.47	213	2264	233	3610
DA-Transformer [12], 2022	12.25	17.08	13.39	19.86	**198**	1575	290	1741
IDMFFN [32], 2023	12.18	19.17	11.89	21.72	205	1819	**206**	3339
KGHM [53], 2023	13.18	13.25	13.54	19.96	251	1131	333	3356
GA-Transformer [54], 2023	11.63	15.99	**11.35**	20.15	215	1133	228	2672
Proposed DA-LSTM,2023	12.62	**13.22**	13.34	**16.25**	263	**842**	360	**1372**

Table 7
Mean and variance of different models.

Methods	RMSE Overall		SCORE Overall	
	$\mu(RMSE)$	$\sigma(RMSE)$	$\mu(SCORE)$	$\sigma(SCORE)$
D-LSTM [10], 2017	21.25	5.25	2798	2244
BS-LSTM [49], 2018	20.99	5.99	3539	3207
BL-CNN [50], 2019	16.75	3.36	1525	1436
RF-LSTM [51], 2020	21.04	3.72	N/A	N/A
AM-LSTM [46], 2021	20.81	6.28	2985	2663
CNN-LSTM [36], 2021	15.60	3.48	1462	1061
AM-BGRU [27], 2021	16.69	3.87	1334	1296
GCU-Transformer [18], 2021	17.59	6.29	N/A	N/A
NTM [52], 2022	20.35	7.24	2820	2615
BIGRU-TSAM [28], 2022	16.10	3.64	1580	1438
DA-Transformer [12], 2022	15.65	3.02	951	710
IDMFFN [32], 2023	16.24	4.30	1392	1303
KGHM [53],2023	14.98	2.88	1268	1254
GA-Transformer [54], 2023	14.78	3.61	1062	1001
Proposed DA-LSTM,2023	**13.86**	**1.41**	**709**	**441**

configuration of the proposed DA-LSTM is only determined on FD004 and shows good generalization ability.

FD001 and FD004 sub-datasets contain one operating condition and six operating conditions respectively. To further demonstrate effectiveness of the proposed method, the prediction results of FD001 and FD004 are visualized. Fig. 10 demonstrate that the proposed model fits the RUL curve well for both sub-datasets.

4.4.2. Comparison for computation quantity

As few works have provided a reference for computational quantity, five models are reproduced for comparison, including two AM based models [12,18] and three similar AM-RNN based models [27,28,46]. Table 8 shows that the number of parameters and FLOPs has a significant improvement over the existing methods. This indicates that the proposed DA-LSTM model combined with the exponential smoothing method is highly parameter efficient. The effect of exponential smoothing is further discussed in Section 4.5. In conclusion, the proposed method can balance the computation quantity and prediction accuracy. The reason is that the C-MAPSS aero-engine degradation dataset is relatively small, which is very common in real industrial applications. However, it is difficult to train a big model with a small dataset. Thus by manually removing unnecessary connections and parameters, it helps mitigate the underfitting problem on big models trained with small datasets, which can lead to high prediction accuracy.

4.5. Data smoothing study

Exponential smoothing is adopted to further train a lightweight model. It can alleviate noise and filter out redundant features, which can make the model mainly focus on the trend of sensor signals. In this section, models with different network hyperparameters are randomly searched to show the relationship between the smooth rate s, loss on the FD004 training set, and the number of parameters.

4.5.1. Effect on computation quantity and convergence speed

Fig. 11(a) reveals the effect of exponential smoothing on the convergence speed. Each loss-epoch line is the averaged result of 50 models. It can be observed that with the increase of s, the convergence speed becomes faster and gradually approaches the model without adopting exponential smoothing ($s = 0$). When $s = 40$, the loss of different epochs can always outperform the original model.

Fig. 11(b) reveals the relationship between the converged loss of the 30th epoch and number of parameters. Each third polynomial curve is the fitting result of 50 different models by the least squares method. With the increase of smooth rate s, a smaller number of parameters is required to reach the same loss. When $s = 40$, models can achieve a balance between the converged loss and the number of parameters, which can outperform the original model.

In conclusion, models with a small value of s perform worse than the original model because ambient noise in the original sensor signal cannot be filtered out thoroughly. However, with the increase of s, the model can gradually focus on the trend of sensor signals instead of the ambient noise. Experiment results show that lightweight models with higher accuracy and fewer parameters are achieved with a suitable smooth rate. On FD001 and FD003 sub-datasets, s is equal to 30. Further, on FD002 and FD004 complex sub-datasets which contain six operating conditions, s is taken as 40 to further eliminate the ambient noise.

4.5.2. Effect on RMSE and SCORE

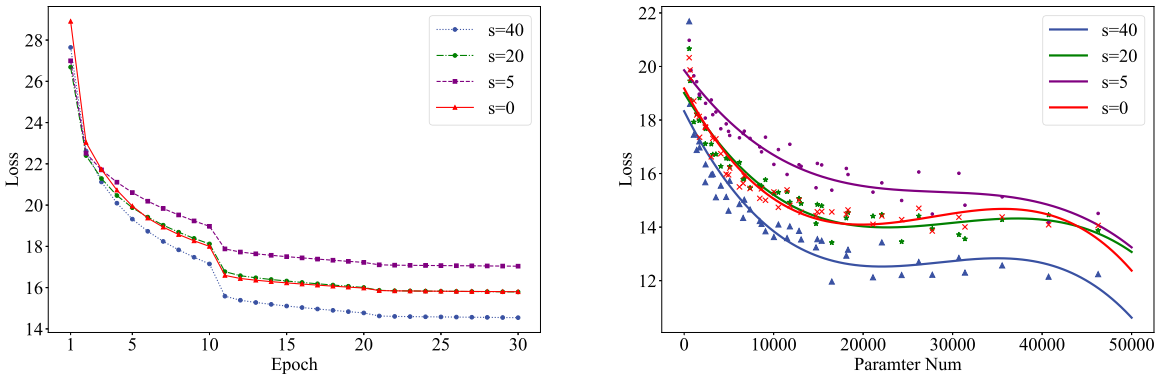
Fig. 12 shows the average of RMSE and SCORE on the validation set. Exponential smoothing can improve the performance of SCORE evidently in the four sub-datasets. Moreover, there is also an evident improvement in RMSE, except for the FD004 dataset, where the RMSE is slightly bigger. In the process of the experiment, we also found that models with similar RMSE may correspond to completely different SCORE and the SCORE fluctuates a lot, which brings great difficulty to train the model. The exponential smoothing method can reduce the volatility of SCORE and keep it at a more stable level, which contributes to training models with low SCORE.

4.6. Model analysis

In this section, ablation study is performed to prove effectiveness of the AOF layer and decoder LSTM network. Then the attention weights of each time step are visualized on the four subdatasets, which contributes to verifying effect of the AEF layer and AOF layer.

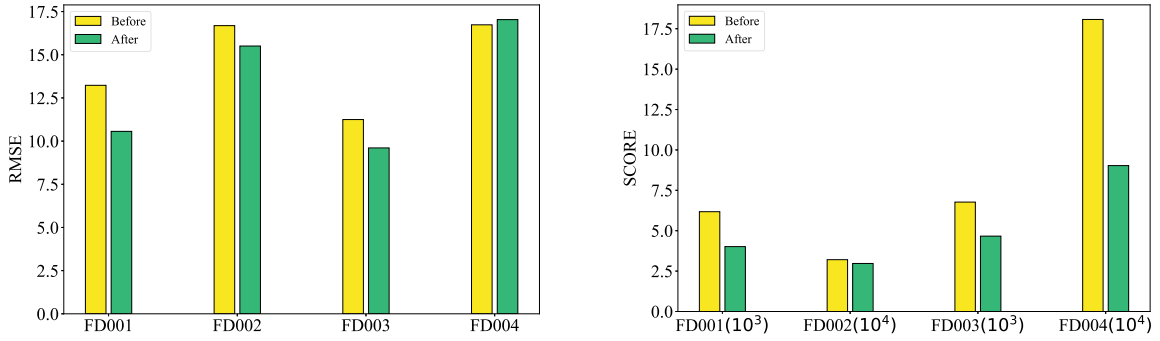
Table 8
Comparison for computation quantity.

Model	Proposed DA-LSTM	AM-LSTM [46]	BIGRU-TSAM [28]	AM-BGRU [27]	DA-Transformer [12]	GCU-Transformer [18]
Parameter num	**3550**	90 061	2 825 443	18 629	116 591	1781937
FLOPs	$\mathbf{1.27 \times 10^5}$	1.06×10^6	1.68×10^8	1.58×10^6	7.44×10^6	6.32×10^7



(a) Effect on convergence speed (b) Effect on parameter num

Fig. 11. Effect on computation quantity and convergence speed.



(a) RMSE (b) SCORE

Fig. 12. Effect on RMSE and SCORE.

Table 9
Abalation study.

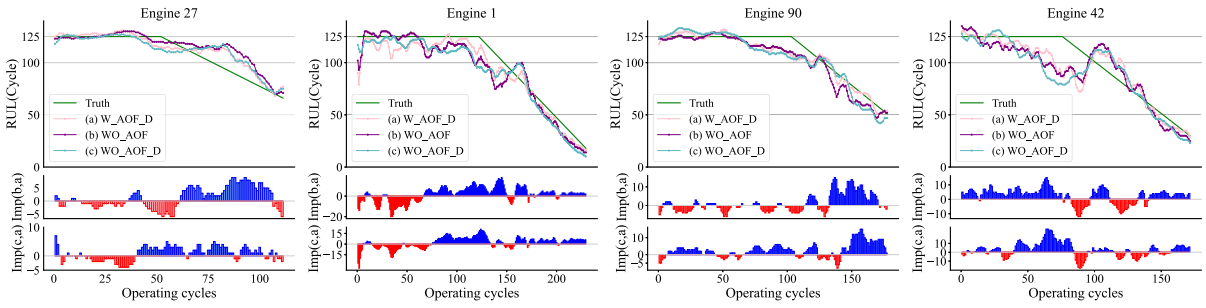
Models	RMSE				SCORE			
	FD001	FD002	FD003	FD004	FD001	FD002	FD003	FD004
(a) W_AOF_D	9.70	14.70	8.98	15.53	3324	22 215	3766	29 778
(b) WO_AOF	10.13	14.90	9.07	16.34	3674	32 074	4311	54 293
(c) WO_AOF_D	10.37	15.34	10.06	16.81	4839	26 291	5043	43 452
$Imp(b,a)_{rmse,score}$	0.43	0.20	0.09	0.81	350	9859	545	24 515
$Imp(c,a)_{rmse,score}$	0.67	0.64	1.08	1.28	1515	4076	1277	13 674

4.6.1. Ablation study

The contribution of each component of the DA-LSTM model is studied on the validation set. We derive three variants, namely (a) DA-LSTM with AOF and decoder LSTM (b) DA-LSTM without AOF (c) DA-LSTM without AOF and decoder LSTM. These three kinds of models are trained on the four subdatasets respectively. Imp means difference of the absolute value. For example, $Imp(b,a)_{rmse}$ means the difference of RMSE absolute value between model (b) and model (a).

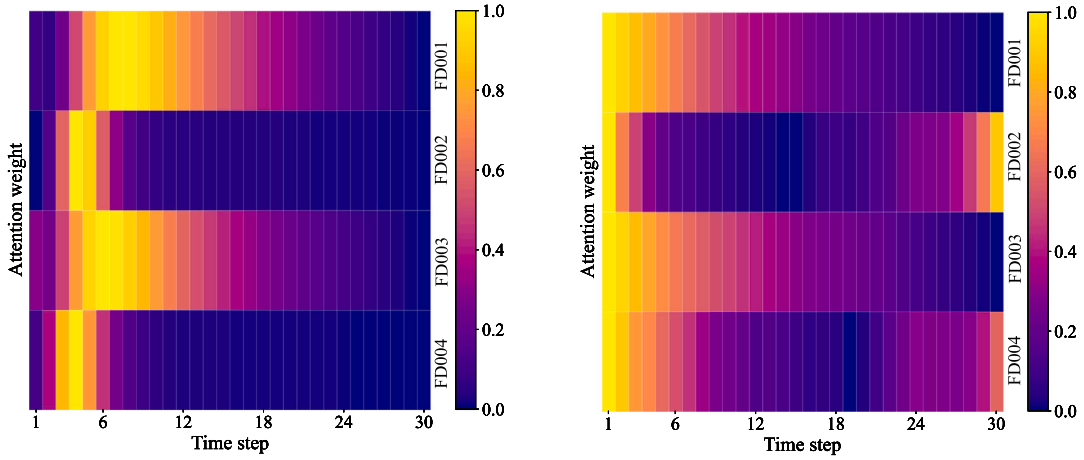
Table 9 presents a comparison among three variants. Model (a) with both AOF and the decoder LSTM achieves the best overall performance. Two conclusions can be further drawn. Firstly, introduction of the decoder LSTM can reduce RMSE and SCORE evidently, which can improve overall performance of the model. Secondly, AOF can reduce the SCORE indicator obviously, especially on the FD002 and FD004 subdataset. This means model with AOF can distinguish early predictions from late predictions, which is useful to develop maintenance plans in advance.

To further test performance of the three models, four engines on the test dataset are randomly selected. The RUL prediction curves are shown in Fig. 13. Note that last part of the operating cycles is not included in the diagram because the complete failure history is not available in the test dataset. The proposed DA-LSTM model performs well on the four engines and is robust on FD002 and FD004, which are complex datasets containing six operating conditions. Moreover, Imp(b,a) and Imp(c,a) are also plotted in the figure. The blue part indicates that model (a) outperforms model (b) or model (c), while the red part means the opposite. In the initial stage, model (a) performs worse than model (b)(c) without AOF or decoder LSTM because the



(a) FD001 (b) FD002 (c) FD003 (d) FD004

Fig. 13. Ablation study.



(a) AEF layer (b) AOF layer

Fig. 14. Attention weight distribution.

engines are in a healthy state and cannot provide adequate degradation information. However, with the increase of operating cycles, the DA-LSTM model can outperform model (b)(c). This means the introduction of AOF and the decoder LSTM is useful to capture SDF when the engine is in a degraded state.

4.6.2. Attention mechanism study

To demonstrate effectiveness of the attention mechanism on aggregate SDF, the normalized attention weights on the four subdatasets are visualized in Fig. 14. The AEF layer assigns more weights at the beginning of the sequence, as is shown in Fig. 14(a). The reason is that these time steps contain the initial degradation state and will affect subsequent degradation trends. However, the AEF layer overlooks the degradation features at time steps 1–2 in the process of feature encoding. To compensate for this problem, the AOF layer assigns more weights to the first two time steps, as is shown in Fig. 14(b). Moreover, for the FD001 and FD003 subdataset which contain only one operating condition, more weights are assigned to the beginning of the sentence. Then the attention weights of both layers decay with time steps gradually. This attention weight distribution can better aggregate the SDF and compensate for the shortcomings of LSTM. Because hidden state at beginning of the sentence will be forgotten due to the chain structure of LSTM. By contrast, for the FD002 and FD004 complex subdataset which contain six operating conditions, the AEF layer pays attention to the beginning of the sequence while the AOF layer assigns more weights to the end of the sequence. The combination of AEF with AOF can make the model have a larger receptive field over the whole input sequence, which is the reason for the high prediction accuracy of DA-LSTM on the FD002 and FD004 subdataset.

5. Conclusion

RUL prediction is the foundation for PHM, which can improve system safety and reduce maintenance costs. A novel DA-LSTM lightweight model is proposed based on exponential smoothing, which can achieve high prediction accuracy with fewer parameters. To better extract SDF and reduce computation quantity, two different AM aggregate the SDF parallelly. The main contributions can be summarized as follows: (1) The first LSTM network can compensate for the weakness of AM in modeling SDF. The second one is decomposed into a single time step to reduce computation quantity and predict RUL. (2) The introduction of the exponential smoothing method can effectively alleviate ambient noise in the original sensor signal. With an appropriate smooth rate, models can achieve higher accuracy with fewer parameters. (3) Ablation studies, scoring function studies, and AM studies are carried out to verify the effectiveness of different model components. Extensive experiments are conducted on the CMAPSS dataset and compared with other SOTA models. The result shows the superiority of our proposed model in prediction accuracy and computation quantity. It is promising to be deployed on edge computing devices with limited computing resources and can provide a reference for future research.

In the future, we plan to design a derivable loss function to find faults as early as possible because late predictions will cause more damage to the system compared with early predictions.

CRediT authorship contribution statement

Jiayu Shi: Writing – original draft, Visualization, Validation, Methodology, Investigation, Conceptualization. **Jingshu Zhong:** Writing – review & editing, Investigation. **Yuxuan Zhang:** Writing

– review & editing, Methodology. **Bin Xiao:** Writing – review & editing. **Lei Xiao:** Writing – review & editing, Conceptualization. **Yu Zheng:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The link to my code at github is shared in the manuscript file.

Acknowledgment

This work is sponsored by National Key R&D Program of China [No. 2020YFB1710802] and Shanghai Key lab of Advanced Manufacturing Environment.

References

[1] Vogl GW, Weiss BA, Helu M. A review of diagnostic and prognostic capabilities and best practices for manufacturing. J Intell Manuf 2019;30(1):79–95. http://dx.doi.org/10.1007/s10845-016-1228-8.

[2] Cui W. A state-of-the-art review on fatigue life prediction methods for metal structures. J Mar Sci Tech-Japan 2002;7(1):43–56. http://dx.doi.org/10.1007/s007730200012.

[3] Zhao F, Tian Z, Zeng Y. Uncertainty quantification in gear remaining useful life prediction through an integrated prognostics method. Ieee Trans Reliab 2013;62(1):146–59. http://dx.doi.org/10.1109/TR.2013.2241216.

[4] Yu W, Mechefske CK, Timusk M. A new dynamic model of a cylindrical gear pair with localized spalling defects. Nonlinear Dynam 2018;91(4):2077–95. http://dx.doi.org/10.1007/s11071-017-4003-2.

[5] Ge C, Zhu Y, Di Y. Hybrid degradation equipment remaining useful life prediction oriented parallel simulation considering model soft switch. In: Franco L, editor. Comput Intell Neurosci 2019;2019:9179870. http://dx.doi.org/10.1155/2019/9179870.

[6] Si X-S, Wang W, Hu C-H, Zhou D-H. Remaining useful life estimation–A review on the statistical data driven approaches. Eur J Oper Res 2011;213(1):1–14. http://dx.doi.org/10.1016/j.ejor.2010.11.018.

[7] Liao L, Köttig F. A hybrid framework combining data-driven and model-based methods for system remaining useful life prediction. Appl Soft Comput 2016;44:191–9. http://dx.doi.org/10.1016/j.asoc.2016.03.013.

[8] Soualhi A, Medjaher K, Zerhouni N. Bearing health monitoring based on Hilbert–Huang transform, support vector machine, and regression. Ieee Trans Instrum Meas 2014;64(1):52–62. http://dx.doi.org/10.1109/TIM.2014.2330494.

[9] Li Z, Goebel K, Wu D. Degradation modeling and remaining useful life prediction of aircraft engines using ensemble learning. J Eng Gas Turb Power 2019;141(4):041008. http://dx.doi.org/10.1115/1.4041674.

[10] Zheng S, Ristovski K, Farahat A, Gupta C. Long short-term memory network for remaining useful life estimation. In: 2017 IEEE international conference on prognostics and health management. 2017, p. 88–95. http://dx.doi.org/10.1109/ICPHM.2017.7998311.

[11] Shi Z, Chehade A. A dual-LSTM framework combining change point detection and remaining useful life prediction. Reliab Eng Syst Safe 2021;205:107257. http://dx.doi.org/10.1016/j.ress.2020.107257.

[12] Liu L, Song X, Zhou Z. Aircraft engine remaining useful life estimation via a double attention-based data-driven architecture. Reliab Eng Syst Safe 2022;221:108330. http://dx.doi.org/10.1016/j.ress.2022.108330.

[13] Wei Y, Wu D. Prediction of state of health and remaining useful life of lithium-ion battery using graph convolutional network with dual attention mechanisms. Reliab Eng Syst Safe 2023;230:108947. http://dx.doi.org/10.1016/j.ress.2022.108947.

[14] Chen Y, Peng G, Zhu Z, Li S. A novel deep learning method based on attention mechanism for bearing remaining useful life prediction. Appl Soft Comput 2020;86:105919. http://dx.doi.org/10.1016/j.asoc.2019.105919.

[15] Devlin J, Chang M-W, Lee K, Toutanova K. BERT: Pre-training of deep bidirectional transformers for language understanding. 2018, http://dx.doi.org/10.48550/arXiv.1810.04805, arXiv:1810.04805.

[16] Zeng A, Liu X, Du Z, Wang Z, Lai H, Ding M, Yang Z, Xu Y, Zheng W, Xia X, Tam WL, Ma Z, Xue Y, Zhai J, Chen W, Zhang P, Dong Y, Tang J. GLM-130b: An open bilingual pre-trained model. 2022, http://dx.doi.org/10.48550/arXiv.2210.02414, arXiv:2210.02414.

[17] Li X, Jiang H, Liu Y, Wang T, Li Z. An integrated deep multiscale feature fusion network for aeroengine remaining useful life prediction with multisensor data. Knowl-Based Syst 2022;235:107652. http://dx.doi.org/10.1016/j.knosys.2021.107652.

[18] Mo Y, Wu Q, Li X, Huang B. Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit. J Intell Manuf 2021;32(7):1997–2006. http://dx.doi.org/10.1007/s10845-021-01750-x.

[19] Ren L, Liu Y, Wang X, Lü J, Deen MJ. Cloud–edge-based lightweight temporal convolutional networks for remaining useful life prediction in iiot. Ieee Internet Things 2020;8(16):12578–87. http://dx.doi.org/10.1109/JIOT.2020.3008170.

[20] Hsu H-Y, Srivastava G, Wu H-T, Chen M-Y. Remaining useful life prediction based on state assessment using edge computing on deep learning. Comput Commun 2020;160:91–100. http://dx.doi.org/10.1016/j.comcom.2020.05.035.

[21] Ren L, Wang T, Jia Z, Li F, Han H. A lightweight and adaptive knowledge distillation framework for remaining useful life prediction. Ieee Trans Ind Inform 2022;19(8):9060–70. http://dx.doi.org/10.1109/TII.2022.3224969.

[22] Yu W, Kim IY, Mechefske C. Remaining useful life estimation using a bidirectional recurrent neural network based autoencoder scheme. Mech Syst Signal Process 2019;129:764–80. http://dx.doi.org/10.1016/j.ymssp.2019.05.005.

[23] Chen D, Hong W, Zhou X. Transformer network for remaining useful life prediction of lithium-ion batteries. Ieee Access 2022;10:19621–8. http://dx.doi.org/10.1109/ACCESS.2022.3151975.

[24] Zhang Z, Song W, Li Q. Dual-aspect self-attention based on transformer for remaining useful life prediction. Ieee Trans Instrum Meas 2022;71:1–11. http://dx.doi.org/10.1109/TIM.2022.3160561.

[25] Hu W, Zhao S. Remaining useful life prediction of lithium-ion batteries based on wavelet denoising and transformer neural network. Front Energy Res 2022;10:1134. http://dx.doi.org/10.3389/fenrg.2022.969168.

[26] Zhao L, Zhu Y, Zhao T. Deep learning-based remaining useful life prediction method with transformer module and random forest. Mathematics 2022;10(16):2921. http://dx.doi.org/10.3390/math10162921.

[27] Liu H, Liu Z, Jia W, Lin X. Remaining useful life prediction using a novel feature-attention-based end-to-end approach. Ieee Trans Ind Inform 2020;17(2):1197–207. http://dx.doi.org/10.1109/TII.2020.2983760.

[28] Zhang J, Jiang Y, Wu S, Li X, Luo H, Yin S. Prediction of remaining useful life based on bidirectional gated recurrent unit with temporal self-attention mechanism. Reliab Eng Syst Safe 2022;221:108297. http://dx.doi.org/10.1016/j.ress.2021.108297.

[29] Tseng S-H, Tran K-D. Predicting maintenance through an attention long short-term memory projected model. J Intell Manuf 2023;1–18. http://dx.doi.org/10.1007/s10845-023-02077-5.

[30] Xiang S, Qin Y, Zhu C, Wang Y, Chen H. LSTM networks based on attention ordered neurons for gear remaining life prediction. Isa T 2020;106:343–54. http://dx.doi.org/10.1016/j.isatra.2020.06.023.

[31] Ragab M, Chen Z, Wu M, Kwoh C-K, Yan R, Li X. Attention-based sequence to sequence model for machine remaining useful life prediction. Neurocomputing 2021;466:58–68. http://dx.doi.org/10.1016/j.neucom.2021.09.022.

[32] Zhang J, Li X, Tian J, Luo H, Yin S. An integrated multi-head dual sparse self-attention network for remaining useful life prediction. Reliab Eng Syst Safe 2023;233:109096. http://dx.doi.org/10.1016/j.ress.2023.109096.

[33] Deng W, Nguyen KT, Gogu C, Morio J, Medjaher K. Physics-informed lightweight temporal convolution networks for fault prognostics associated to bearing stiffness degradation. In: PHM Society European Conference, Vol. 7. (1):2022, p. 118–25. http://dx.doi.org/10.36001/phme.2022.v7i1.3365.

[34] Kayode O, Tosun AS. Lirul: A lightweight lstm based model for remaining useful life estimation at the edge. In: 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC), Vol. 2. 2019, p. 177–82. http://dx.doi.org/10.1109/COMPSAC.2019.10203.

[35] Kumar MP, Gao Z-J, Chen K-C. Time series-based sensor selection and lightweight neural architecture search for RUL estimation in future industry 4.0. Ieee J Emerg Sel Top Circuits Syst 2023;13(2):514–23. http://dx.doi.org/10.1109/JETCAS.2023.3248642.

[36] Mo H, Custode LL, Iacca G. Evolutionary neural architecture search for remaining useful life prediction. Appl Soft Comput 2021;108:107474. http://dx.doi.org/10.1016/j.asoc.2021.107474.

[37] Sun J, Zhang X, Wang J. Lightweight bidirectional long short-term memory based on automated model pruning with application to bearing remaining useful life prediction. Eng Appl Artif Intel 2023;118:105662. http://dx.doi.org/10.1016/j.engappai.2022.105662.

[38] Jiang G, Zhou W, Chen Q, He Q, Xie P. Dual residual attention network for remaining useful life prediction of year=2022, bearings. Measurement 199:111424. http://dx.doi.org/10.1016/j.measurement.2022.111424.

[39] Luong M-T, Pham H, Manning CD. Effective approaches to attention-based neural machine translation. 2015, http://dx.doi.org/10.48550/arXiv.1508.04025, arXiv:1508.04025.

[40] Saxena A, Goebel K, Simon D, Eklund N. Damage propagation modeling for aircraft engine run-to-failure simulation. In: 2008 International conference on prognostics and health management. 2008, p. 1–9. http://dx.doi.org/10.1109/PHM.2008.4711414.

[41] Huang C-G, Huang H-Z, Li Y-F. A bidirectional LSTM prognostics method under multiple operational conditions. Ieee Trans Ind Electron 2019;66(11):8792–802. http://dx.doi.org/10.1109/TIE.2019.2891463.
[42] Zhang C, Lim P, Qin AK, Tan KC. Multiobjective deep belief networks ensemble for remaining useful life estimation in prognostics. Ieee Trans Neural Netw Learn Syst 2016;28(10):2306–18. http://dx.doi.org/10.1109/TNNLS.2016.2582798.
[43] Heimes FO. Recurrent neural networks for remaining useful life estimation. In: 2008 International conference on prognostics and health management. 2008, p. 1–6. http://dx.doi.org/10.1109/PHM.2008.4711422.
[44] Sateesh Babu G, Zhao P, Li X-L. Deep convolutional neural network based regression approach for estimation of remaining useful life. In: International conference on database systems for advanced applications. Springer; 2016, p. 214–28. http://dx.doi.org/10.1007/978-3-319-32025-0_14.
[45] Lei Y, Li N, Guo L, Li N, Yan T, Lin J. Machinery health prognostics: A systematic review from data acquisition to RUL prediction. Mech Syst Signal Process 2018;104:799–834. http://dx.doi.org/10.1016/j.ymssp.2017.11.016.
[46] Chen Z, Wu M, Zhao R, Guretno F, Yan R, Li X. Machine remaining useful life prediction via an attention-based deep learning approach. Ieee Trans Ind Electron 2020;68(3):2521–31. http://dx.doi.org/10.1109/TIE.2020.2972443.
[47] Werbos PJ. Backpropagation through time: what it does and how to do it. Proc IEEE 1990;78(10):1550–60. http://dx.doi.org/10.1109/5.58337.
[48] Greff K, Srivastava RK, Koutník J, Steunebrink BR, Schmidhuber J. LSTM: A search space odyssey. Ieee Trans Neural Netw Learn Syst 2016;28(10):2222–32. http://dx.doi.org/10.1109/TNNLS.2016.2582924.
[49] Liao Y, Zhang L, Liu C. Uncertainty prediction of remaining useful life using long short-term memory network based on bootstrap method. In: 2018 Ieee international conference on prognostics and health management. (Icphm), IEEE; 2018, p. 1–8.
[50] Liu H, Liu Z, Jia W, Lin X. A novel deep learning-based encoder-decoder model for remaining useful life prediction. In: 2019 International joint conference on neural networks. (IJCNN), 2019, p. 1–8. http://dx.doi.org/10.1109/IJCNN.2019.8852129.
[51] Tang J, Xiao L. The improvement of remaining useful life prediction for aero-engines by classification and deep learning. In: 2020 11th International conference on prognostics and system health management. (PHM-2020 Jinan), 2020, p. 130–6. http://dx.doi.org/10.1109/PHM-Jinan48558.2020.00030.
[52] Falcon A, D'Agostino G, Lanz O, Brajnik G, Tasso C, Serra G. Neural turing machines for the remaining useful life estimation problem. Comput Ind 2022;143:103762. http://dx.doi.org/10.1016/j.compind.2022.103762.
[53] Li Y, Chen Y, Hu Z, Zhang H. Remaining useful life prediction of aero-engine enabled by fusing knowledge and deep learning models. Reliab Eng Syst Safe 2023;229:108869. http://dx.doi.org/10.1016/j.ress.2022.108869.
[54] Mo H, Iacca G. Evolutionary neural architecture search on transformers for RUL prediction. Mater Manuf Proces 2023;1–18. http://dx.doi.org/10.1080/10426914.2023.2199499.